



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduzione a Javascript

Temi trasversali (III parte)

Fabio Vitali

Corso di laurea in Informatica

Alma Mater – Università di Bologna

Oggi parleremo di...

Javascript

- Sintassi base (parte I)
- Javascript client-side (parte II)
- Sintassi avanzata (parte III)

Temi trasversali

- Navigazione sul DOM
- AJAX
- Programmazione asincrona
- Modularizzazione del codice
- Interpolazione
- ***Type checking e Typescript***
- ***Routing***
- ***Binding mono e bi-direzionale***





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Type checking e Typescript

Il problema

```
let el = document.getElementById('someId')  
el.classList.add('red')
```

Questo codice non è sempre eseguibile

- Se l'elemento con id *someId* non esiste, viene sollevata un'eccezione e l'esecuzione viene interrotta
- Qualunque istruzione successiva nello stesso blocco non sarà valutata, e senza una corretta gestione delle eccezioni viene sospesa l'esecuzione dell'intero script.

Oppure:

```
function add(a, b) {  
  return a+b  
}  
  
let primo = add(3, 4)  
let secondo = add(3, '4')
```

Quanto vale primo? Quanto vale secondo?



I tipi in JavaScript

Il linguaggio JavaScript presenta un sistema di tipi dinamico, in cui cioè la tipizzazione viene eseguita durante l'esecuzione del codice. Questo rende difficile il debugging, perché non è possibile eseguire un'analisi statica dei tipi... o per lo meno è talmente problematica da non essere praticabile.

Vi sono design pattern, inoltre, in cui essa è di fatto impossibile.

Se su progetti di piccola scala questo aspetto potrebbe non essere problematico, con l'aumento delle righe di codice il problema diventa sempre più pressante; è molto facile, inoltre, far sì che il codice manifesti comportamenti diversi a causa dei parametri utilizzati per invocare le funzioni.

Tutto questo, però, può essere valutato solo a runtime... Eseguendo il codice in ogni possibile scenario, seguendo tutte le sue possibili ramificazioni... E presto diventa non sostenibile.



Typescript

Linguaggio open source sviluppato da Microsoft a partire dal 2012, proprio per rendere più agevole l'analisi statica dei tipi in JavaScript.

Typescript è un super-set di JavaScript, perciò ogni costrutto valido in JavaScript è valido in Typescript;

Vengono aggiunti costrutti per l'annotazione dei tipi e un sistema di type-inference che consente di conoscere (staticamente) il tipo di una variabile.

Un compilatore si occupa di compilare (*transpiling*) i sorgenti Typescript in JavaScript vero e proprio, eseguendo nel contempo l'analisi statica dei tipi



In Typescript

```
let el = document.getElementById('someId');  
el.classList.add('red');
```

compile time error: 'el' is possibly 'null'.

document.getElementById restituisce un valore di tipo `HTMLElement | null`, e questo tipo quindi potrebbe essere null.

L'analisi statica permette di identificare una possibile sorgente di errori.

```
function add(a:number, b:number): number {  
    return a+b;  
}
```

```
let primo = add(3, 4);  
let secondo = add(3, '4');
```

compile time error: Argument of type 'string' is not assignable to parameter of type 'number'.

Il primo argomento deve essere un intero, e invece viene passata una stringa alla seconda invocazione di add().



Estensioni di Typescript

- Annotazioni implicite di tipo (per inferenza)

```
const el = 'Hello world' ;    // el è di tipo string
let num = 16;                  // num è di tipo number
```

- Annotazioni esplicite di tipo

```
const el: string = 'Hello world' ;
let num: number = 16;
```

- Dichiarazione di tipo in funzioni e array

```
function add(a:number, b:number): number {
    return a+b;
}
let arrayOfNumbers: number[] = [1,2,3,4]
```

- Creazione di nuovi tipi (generics)

```
type arrayOfStrings = Array<string>;
let aos: arrayOfStrings = ['a', 'b', 'c'];
```


Estensioni di Typescript

- Tipo generico (any)

```
let abc: any = { a: 'x', b: 15, c: new Date() };
```

- Interfacce

- riporta come errore campi non specificati nell'interfaccia

```
interface Person {  
  name: string;  
  age: number;  
}  
let user: Person = {  
  name: "Mario",  
  age: 21  
};
```



Estensioni di Typescript

- Tipo generico (any)

```
let abc: any = { a: 'x', b: 15, c: new Date() };
```

- Interfacce

– riporta come errore campi non specificati nell'interfaccia

```
interface Person {  
  name: string;  
  age: number;  
}  
let user: Person = {  
  name: "Mario",  
  age: 21,  
  alias: "Super Mario"  
};
```

Compile time error: Object literal may only specify known properties, and 'alias' does not exist in type 'Person'.



Estensioni di Typescript

- Dichiarazioni di classi con private, public

```
class Person {  
  private fiscalcode: string;  
  public name: string;  
  public age: number;  
  
  constructor(name: string, age: number, cf:string) {  
    this.name = name;  
    this.age = age;  
    this.fiscalcode = cf  
  }  
  toString(): string {  
    return `${this.name} (cf: ${this.fiscalcode}) ha ${this.age} anni.`  
  }  
}  
  
let mario = new Person('Mario', 21, "BTMRA97C12A944T") ;  
let lucia = new Person('Lucia', 23, "MSVLCU02D48A944S") ;  
alert(lucia.toString())
```

Lucia (cf: MSVLCU02D48A944S) ha 23 anni.



Estensioni di Typescript

- Estensioni di classi e in più static, readonly

```
class Employee extends Person {
  static numOfEmployee: number = 0;
  readonly cmp: string = "ACME";
  private id: number;

  constructor(name: string, age: number, cf:string) {
    super(name, age, cf);
    Employee.numOfEmployee++
    this.id = Employee.numOfEmployee;
  }
  toString(): string {
    return `${this.name} (matricola: ${this.cmp}-${this.id}), ${this.age}.`
  }
}

let mario = new Employee('Mario', 21, "BTTMRA97C12A944T") ;
let lucia = new Employee('Lucia', 23, "MSVLCU02D48A944S") ;
alert(lucia.toString())
```

Lucia (matricola: ACME-2), 23.

Estensioni di Typescript

- Decoratori (forse prossima feature di ECMAScript?)
 - funzioni introdotte da @ e associate a classi, metodi, proprietà o parametri.
 - funzioni wrapper che precedono e seguono le invocazioni relative e possono cambiarle.

```
class Person {
  private fiscalcode: string;
  public name: string;
  public age: number;
  constructor(name: string, age: number, cf:string) {
    this.name = name;
    this.age = age;
    this.fiscalcode = cf
  }
  @check
  toString(): string {
    return `${this.name} (cf: ${this.fiscalcode}), ${this.age} anni.`
  }
  @check
  hello() { alert( `Hello, ${this.name}.` ) }
}

function check( target: any, key: string, descr: PropertyDescriptor):
PropertyDescriptor {
  const original = descr.value; // salvo il metodo originale
  descr.value = function (...args: any[]) { // creo un metodo wrapper
    console.log("sono qui: " + key); // che mostra il nome del metodo,
    return original.apply(this, args); // e poi chiama l'originale
  };
}
```



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Routing

Routing

- Con routing intendiamo l'assegnazione di un URI diverso ad ogni stato dell'applicazione web (cioè ad ogni selezione di contenuto differente).
- Con l'approccio LAMP il problema del routing è molto semplice: ad ogni pagina HTML e/o ogni script server-side è automaticamente associato un URI (file system routing) e non ci sono complicazioni di sorta.
- Con le applicazioni Ajax e con Node le cose si complicano. Possiamo dividere la questione in due sottoproblemi profondamente diversi:
 - Routing server-side
 - Routing client-side



Routing server-side

Nel routing server-side ipotizziamo che

- il browser stia eseguendo navigazione (cioè si aspetti di ricevere un'intera pagina HTML come risposta alla richiesta HTTP), e
- sia compito dell'application logic server-side decidere quale contenuto mandare

A decidere quali contenuti mandare giocano tre fattori:

- il metodo HTTP della richiesta (GET, POST, PUT, DELETE)
- l'URI della richiesta
 - protocollo ([HTTP](#), [HTTPS](#))
 - ~~dominio (e.g. [www.site.com:80](#))~~
 - path (e.g., [/dir1/dir2/nome](#))
 - query (e.g., [?var1=value1&var2=value2](#))
 - ~~fragment (e.g. [#identifier1](#))~~
- gli header della richiesta

*Non di competenza
del router*



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Routing server-side

express

Express.js è la libreria più importante di Node.js, ha come scopo specifico creare un meccanismo di routing server-side.

```
// file index.js
var express = require('express');
var app = express();

var routes = require('./routes.js');
app.use('/', routes);
app.listen(8000);
```

```
// file routes.js
var express = require('express');
var router = express.Router();

router.get('/', function(req, res){
  res.send('<p>This is the home page</p>');
});
router.get('/about', function(req, res){
  res.send('<p>This is the about page</p>');
});
router.post('/contacts', function(req, res){
  res.send('<p>This is the contacts page</p>');
});
module.exports = router;
```

Routing client-side

Single Page Application

- Parliamo di routing client-side quando non contiamo sulla navigazione via server per passare da un contenuto all'altro. In particolare la questione si presenta con le cosiddette *SPA (Single Page Application)*.
- Una SPA è un'applicazione web composta da un unico documento HTML il cui contenuto visibile è generato (anche solo in parte) da un'applicazione Javascript che possiede all'avvio, oppure carica via via, il contenuto dal server usando script per decidere di volta in volta cosa visualizzare a schermo.
- Poiché la pagina HTML è una sola, se lo script non fa niente, l'URI non cambia mai qualunque sia il contenuto visualizzato. Questo approccio garantisce stile, interattività e semplicità di progettazione, ma ha effetti nefasti per navigazione, bookmark, back/forward, SEO, website analytics, sicurezza, etc.
 - Client-side rendering (CSR - l'approccio Ajax tradizionale)
 - Server-side rendering (SSR)
 - Static Site Generators (SSG)



Routing client-side

CSR – SSR – SSG

Client-side rendering (CSR)

- La pagina HTML ricevuta contiene il codice JS e CSS ma è vuota oppure contiene contenuto iniziale generico (benvenuto, grafica di caricamento, ecc.)
- Appena inizializzato lo script procede a richiedere contenuto via XHR e ad approntare il DOM secondo necessità.
- Ogni volta che bisogna visualizzare nuovo contenuto (tipicamente riceve via XHR dati strutturati in JSON) lo script crea client-side delle modifiche al DOM e quindi otteniamo la nuova visualizzazione.

Server-side rendering (SSR)

- Lo stato della computazione è mantenuto server-side. La pagina HTML iniziale contiene del contenuto iniziale generico e degli script minimali di interazione con il server.
- Ogni volta che bisogna visualizzare nuovo contenuto lo script manda i parametri dell'operazione al server, che genera immediatamente un nuovo HTML che viene mandato in risposta al browser.

Static Site Generators (SSG)

- Lo stato della computazione è mantenuto server-side, e in fase di compilazione lo script crea in maniera statica un alto numero di pagine HTML una per ogni possibile stato del contenuto. Durante la navigazione non c'è nessuna computazione, ma solo la scelta di quale pagina HTML spedire al browser.



Routing client-side

Location e History

- L'oggetto predefinito `window.location` contiene le informazioni sull'URI della pagina attualmente visualizzata.
- Se non navigo, il valore non cambierà mai. Quindi anche se il contenuto della pagina cambia in seguito a modifiche al DOM, la location sarà sempre la stessa e SEO, i pulsanti back e forward e i bookmark non registreranno lo stato attuale dell'applicazione.
- L'oggetto predefinito `window.history` contiene lo stack degli URI visitati in questa sessione in questa finestra del browser.
- Se non navigo, lo stack non cambierà mai.
- E' necessario gestire esplicitamente location e history per avere degli URI e degli stati di history navigabili e utilizzabili da bookmark e motori di ricerca.



Routing client-side

History API

- History API fornisce accesso in lettura a scrittura allo stack di history della finestra del browser.
- Oltre a metodi di pura navigazione come `back()`, `forward()` e `go(n)`, permette anche di modificare lo stack stesso inserendo e togliendo nuovi elementi.
- `pushState(state, title, url)`
 - Aggiunge una nuova voce nello stack della history.
 - L'URI cambia senza ricaricare la pagina.
 - I pulsanti Back/Forward permettono di tornare agli stati precedenti.
 - Ogni cambiamento nella pagina può diventare un nuovo stato navigabile.
- `replaceState(state, title, url)`
 - Sostituisce l'ultimo stato della history.
 - L'URI cambia, ma non aumenta la history.
 - Utile per nascondere pagine di avvio, redirect invisibili, aggiornamenti di stato che non devono apparire nella navigazione.



Routing client-side

Routing fatto a mano

```
<!doctype html>
<html>
  <body>
    <nav><ul>
      <li><a href="#" onclick="goto('home')">Home</a></li>
      <li><a href="#" onclick="goto('about')">About</a></li>
      <li><a href="#" onclick="goto('contacts')">Contacts</a></li>
    </nav>
    <main id="content"> </main>
    <footer> Altre cose sempre uguali </footer>
  </body>
</html>
```

```
let routes = {
  home: `<h1>welcome to my home page</h1>
        <p>Some content for the home page</p>`,
  about: `<h1>About this web site</h1>
        <p>It was created by Fabio Vitali @ 2025</p>`,
  contacts: `<h1>Contacts</h1>
            <p>Write to fabio.vitali@unibo.it</p>`
}
```

Routing client-side

Routing fatto a mano

```
const content = document.getElementById('content');

function goto(destination) {
  content.innerHTML = routes[destination];
  let newUri = "/" + destination ;
  window.history.pushState( {path: destination}, "", newUri );
}
```

- `pushState()` aggiunge un nuovo stato allo stack di history.
 - Il primo parametro è un'informazione tecnica qualunque che può servire in seguito per ricostruire lo stato quando l'utente naviga e ricarica la pagina
 - Il secondo parametro non è usato ma non può essere omissso. Va bene una stringa vuota
 - Il terzo parametro è l'URI che vogliamo sia associato a questo stato dello stack, e che verrà visualizzato nella barra dell'indirizzo



Routing client-side

Angular router

- Uso la libreria RouterModule
- Creo un componente per ciascuna route e genero un RoutingModule
- Uso la direttiva **routerLink** nei link invece di **href**

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from '../components/home/home.component';
import { AboutComponent } from '../components/about/about.component';
import { ContactComponent } from '../components/contact/contact.component';
```

```
const routes:Routes = [
  { path: '', redirectTo: 'home' }, //default route
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'contacts', component: ContactComponent }
];
```

```
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {
  <nav>
    <a class="button" routerLink="">Home</a>
    <a class="button" routerLink="about">About</a>
    <a class="button" routerLink="contacts">Contacts</a>
  </nav>
```


Routing client-side

React router

- React non ha routing, ma esiste la libreria molto diffusa React Router
- Creo un componente Route per ciascuna route dentro ad App (il componente di base dell'applicazione)

```
// App.js
import { Routes, Route } from 'react-router-dom';
import Home from './Pages/Home';
import About from './Pages/About';
import Contacts from './Pages/Contacts';
const App = () => {
  return (
    <>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contacts" element={<Contacts />} />
      </Routes>
    </>
  );
};
export default App;
```

Routing client-side

Vue Router

- Vue Router è una libreria separata da Vue
- Fornisce due componenti, RouterView (che mostrerà il contenuto di ciascuna route) e RouterLink (che attiva questa o quella route)
- Anche qui debbo creare un componente per ciascuna route

```
import { createMemoryHistory, createRouter } from 'vue-router'  
import HomeView from './HomeView.vue'  
import AboutView from './AboutView.vue'  
import ContactsView from './ContactsView.vue'
```

```
const [ ...  
  {  
    {  
    {  
  }  
]  
  </nav>  
  <main>  
    <RouterView />  
  </main>  
</template>  
const his  
rou  
})
```

Routing dinamico (o parametrico)

Express

```
app.get('/users/:id', function(req, res) {  
  var identifier = req.params.id;  
  res.render('modules/users/' + identifier + '.html');  
});
```

Angular

```
const routes:Routes = [  
  ...  
  { path: 'users/', component: UserListComponent },  
  { path: 'users/:id', component: UserComponent }  
];
```

disponibile negli script come

```
const route: ActivatedRoute;  
route.params.subscribe(p => {  
  console.log(p.id);  
})
```

React

```
<Routes>  
  <Route path="/users"      element={<UserList />} />  
  <Route path="/users/:id" element={<User />} />  
</Routes>
```

disponibile negli script come

```
const params = useParams();  
console.log(params.id);
```

Vue

```
const routes = [  
  { path: '/users/',    component: UserList },  
  { path: '/users/:id', component: User },  
]
```

disponibile negli script come

```
const route = useRoute();  
console.log(route.params.id);
```

Server-side generation

Server-side rendering

Static web page generation

- I più famosi SSG e SSR sono senza dubbio i fratelli gemelli Next.js (per React) e Nuxt (per Vue)
- Next.js esegue pre-render di default: l'engine prova a eseguire tutto il javascript che riesce prima di consegnare la pagina al browser
- Questo è particolarmente importante quando uso framework a componenti perché il load time dei componenti (*hydration*) può essere lungo.
 - **Static generation**: le pagine HTML sono generate a build time (compilate in HTML) e immediatamente disponibili ad ogni richiesta
 - Utile se il contenuto della pagina non cambia o cambia limitatamente tra utente e durante la visualizzazione della pagina.
 - **Server-side rendering**: le pagine HTML sono generate di volta in volta ad ogni richiesta.
 - Utile se l'input dell'utente è determinante per lo stato finale della pagina
- La scelta può avvenire pagina per pagina ed essere cambiata al volo



Next.js

- Next.js è un framework pensato per React che gestisce automaticamente rendering (SSR, SSG), routing, ottimizzazioni e deployment.
- Trasforma (una parte di) un'applicazione React in un sito tradizionale, veloce, indicizzabile, con pagine già pronte lato server.
 - Routing automatico: ogni file in app/ o pages/ diventa un percorso.
 - API Routes: endpoint server-side nello stesso progetto.
 - Rendering flessibile: SSR, SSG, e anche ISR (rigenerazione ibrida).
 - File-based structure: niente configurazioni iniziali complesse.



Usare Next.js

```
// main.jsx
import React from "react"
import ReactDOM from "react-dom/client"
import { BrowserRouter } from "react-router-dom"
import App from "./App"

ReactDOM.createRoot(document.getElementById("root"))
  .render(
    <BrowserRouter>
      <App />
    </BrowserRouter>
  )
```

Una applicazione
che usa React
come questa...

```
// App.jsx
import { Routes, Route, Link } from "react-router-dom"
import Home from "./Home"
import About from "./About"

export default function App() {
  return (
    <>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </>
  )
}
```

Usare Next.js

```
// pages/_app.jsx
import type { AppProps } from "next/app"

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />
}
```

... con Next.js può diventare...

- Non ci sono più `main.jsx` né `ReactDOM.createRoot`.
- Ogni file in `pages/` e `app/` è una pagina: il routing è automatico.
- Il server può renderizzare le pagine (SSR / SSG) prima di spedirle al browser.
- `_app.jsx` è un file speciale usato per inizializzare le pagine.
- Next.js lo esegue prima di tutte le pagine e permette di settare configurazioni, layout ecc.

```
// pages/index.jsx
import Link from "next/link"

export default function Home() {
  return (
    <>
      <nav>
        <Link href="/">Home</Link>
        <Link href="/about">About</Link>
      </nav>
      <h1>Home page</h1>
    </>
  )
}
```





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Binding

Binding

- Il binding è il collegamento tra dati usati in parti diverse di un programma.
 - L'idea è che se qualcuno cambia il dato in una parte, il nuovo valore viene propagato anche negli altri dati collegati a questo.
 - il metodo `.bind()` delle funzioni Javascript non c'entra niente.
- Il binding può essere
 - monodirezionale (one-way binding): esiste una locazione primaria del dato, che è unica ed è quella che riceve il nuovo valore, e tante locazioni secondarie, a cui il nuovo valore viene propagato automaticamente.
 - bidirezionale (two-way binding): tutte le locazioni del dato sono modificabili e qualunque di essa riceve il nuovo dato lo propaga a tutte le altre locazioni.



Altri nomi per il binding

- ***Binding*** (one-way, two-way)
- ***State management***
- Reactivity
- Observers/observable (*molti usi diversi*)
- Watchable (*molti usi diversi*)
- Publish/subscribe (*pub/sub*)
- Hooks
- Signals
- ...



Model-View-Controller

Model-View-ViewModel

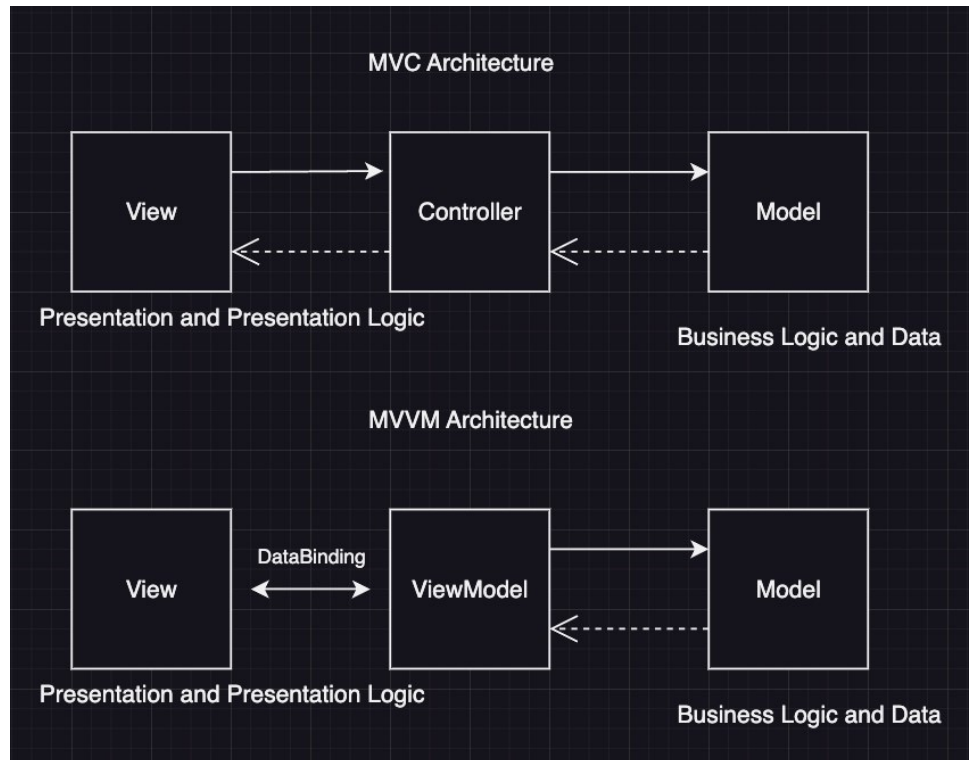
Sono due pattern di programmazione molto importanti alla base di qualunque progettazione delle interfacce.

MVC divide l'interazione tra

- il *Modello*, che rappresenta la natura concettuale del dato
- La Vista, che è un widget che mostra all'utente il dato
- Il Controller, che allinea il modello alla vista e si occupa dell'interazione con l'utente.

MVVM divide l'interazione tra

- il *Modello*, come sopra
- La Vista, come sopra
- Il ViewModel, che collega (binding) elementi della vista alle strutture del modello e lascia che sia il widget a propagare le modifiche al modello.



Un esempio

Pensiamo ad un'applicazione che visualizza i dettagli di vendita di N agenti. Questo database è visualizzato come tabella (una riga per ogni venditore) ma anche come grafico delle vendite mese per mese. L'utente può dinamicamente assegnare una commissione agli agenti.

- **Modello**: un DB con un record per ogni agente: nome, mesi.
- **Vista 1**: una tabella, un'intestazione, una riga per ogni record, una colonna per ogni campo.
- **Vista 2**: un grafico a barre per mese con la somma di tutti gli importi e le commissioni.
- **Controller**: uno script che viene chiamato all'inizio per creare le due viste, e ogni volta che l'utente modifica valore, ricalcola l'importo e i totali per mese.
- **ViewModel**: uno script che viene chiamato all'inizio per creare le due viste e che associa al campo di input uno script che aggiorna il modello ogni volta che il campo cambia valore.



Un esempio

- Ma cosa succede quando si moltiplicano e si incrociano i bind tra dati del modello e oggetti di visualizzazione?
- Il framework deve andare su e giù per il DOM alla ricerca degli oggetti da aggiornare ogni volta che cambia un valore, e le dipendenze possono essere tanti e complesse.
- Calcoliamo un po' di totali: di ogni venditore calcoliamo il totale vendite, di ogni mese calcoliamo il totale vendite, e calcoliamo i due totali complessivi che debbono coincidere. Poi calcoliamo la commissione da erogare a ciascun venditore pari all' $X\%$ della vendita.
- **L'utente della vista può cambiare sia i dati mese x venditore, sia la percentuale della commissione.**
- In pratica, per calcolare il totale netto debbo aspettare di avere il totale lordo che dipende dal totale di ogni colonna e/o riga, che dipende dal contenuto di ogni singola cella della colonna/riga.
- Le dipendenze sono molto e molto complesse. Inoltre debbo evitare di avere cicli nelle dipendenze che impedirebbero il calcolo.

Come si può fare?



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Un esempio

	A	B	C	D	E	F	G	H
1	Commissione	5%						
2								
3	Vendite	Gennaio	Febbraio	Marzo	Aprile	Maggio	Giugno	Totali
4	Alice	40	20	83	60	21	25	249
5	Bruno	40	77	83	22	79	87	388
6	Carla	86	72	77	93	18	35	381
7	Davide	91	73	11	43	11	52	281
8	Totali	257	242	254	218	129	199	1299
9	Ricavi	12,85	12,1	12,7	10,9	6,45	9,95	64,95



Dipendenze multiple

	A	B	C	D	E	F	G	H
1	Commissione	5%						
2								
3	Vendite	Gennaio	Febbraio	Marzo	Aprile	Maggio	Giugno	Totali
4	Alice	40	20	83	60	21	25	=SUM(B4:G4)
5	Bruno	40	77	83	22	79	87	=SUM(B5:G5)
6	Carla	86	72	77	93	18	35	=SUM(B6:G6)
7	Davide	91	73	11	43	11	52	=SUM(B7:G7)
8	Totali	=sum(B4:B7)	=sum(C4:C7)	=sum(D4:D7)	=sum(E4:E7)	=sum(F4:F7)	=sum(G4:G7)	=sum(H4:H7)
9	Ricavi	=B8*\$B\$1	=C8*\$B\$1	=D8*\$B\$1	=E8*\$B\$1	=F8*\$B\$1	=G8*\$B\$1	=H8*\$B\$1

- La soluzione viene dai fogli elettronici come MS Excel, che si creano un albero di dipendenze esplicite, e appena notano un cambiamento in uno degli elementi sorgenti propagano gli aggiornamenti a tutti i figli del nodo.
- Inoltre a compile time sono in grado di vedere immediatamente se ci sono delle dipendenze circolari e segnarle come errori senza aspettare di calcolare alcunché.

	A	B	C	D	E	F	G	H
1	Commissione	=H9*5%						
2								
3	Vendite	Gennaio			Aprile	Maggio	Giugno	Totali
4	Alice	40		83	60	21	25	=SUM(B4:G4)
5	Bruno	40		83	22	79	87	=SUM(B5:G5)
6	Carla	86	72	77	93	18	35	=SUM(B6:G6)
7	Davide	91	73	11	43	11	52	=SUM(B7:G7)
8	Totali	=sum(B4:B7)	=sum(C4:C7)	=sum(D4:D7)	=sum(E4:E7)	=sum(F4:F7)	=sum(G4:G7)	=sum(H4:H7)
9	Ricavi	=B8*\$B\$1	=C8*\$B\$1	=D8*\$B\$1	=E8*\$B\$1	=F8*\$B\$1	=G8*\$B\$1	=H8*\$B\$1

Error

Circular dependency detected.
To resolve with iterative
calculation, see File > Settings.

Data binding fatto a mano (MVC)

Un campo editabile (*modifiable*), uno span che riceve le modifiche (*passive*)

```
<input id="edit">  
<span id="show"></span>
```

```
function Bound(element1, element2, startingData) {  
    this.modifiable = element1;  
    this.passive = element2  
    this.model = startingData;  
    this.modifiable.value = this.model;  
    this.modifiable.addEventListener("change", this.change.bind(this));  
    this.modifiable.addEventListener("keyup", this.keyup.bind(this));  
    this.modifiable.dispatchEvent(new Event("change"))  
}  
Bound.prototype.change = function (event) {  
    this.model = event.target.value;  
    this.modifiable.value = this.model;  
    this.passive.innerHTML = this.model;  
}  
Bound.prototype.keyup = function (value) {  
    this.change(value) ;  
}  
var bound = new Bound(  
    document.getElementById("edit"),  
    document.getElementById("show"),  
    "inizio");
```

<https://jsfiddle.net/91Lpocz/>

Data binding fatto a mano (MVVM)

Un campo editabile (*modifiable*), uno span che riceve le modifiche (*passive*)

```
<input id="edit">  
<span id="show"></span>
```

```
function Bound(element1, element2, startingData) {  
    this.modifiable = element1;  
    this.passive = element2;  
  
    this.modifiable.value = startingData;  
    this.modifiable.addEventListener("change", this.change.bind(this));  
    this.modifiable.addEventListener("keyup", this.keyup.bind(this));  
    this.modifiable.dispatchEvent(new Event("change"))  
}  
Bound.prototype.change = function (event) {  
    let v = event.target.value;  
    this.modifiable.value = v;  
    this.passive.innerHTML = v;  
};  
Bound.prototype.keyup = function (value) {  
    this.change(value)  
};  
var bound = new Bound(  
    document.getElementById("edit"),  
    document.getElementById("show"),  
    "inizio");
```

<https://jsfiddle.net/vn815fx9/>

Angular

- Ogni binding è composto di tre parti:
 - un *valore* oggetto del collegamento, disponibile all'interno del contesto
 - un *target* nel DOM che riceverà il valore suddetto
 - Un *template* che formatterà il valore in maniera appropriata
- Angular distingue quattro tipi di data binding
 - Interpolazioni di frammenti di HTML

```
<ul>  
  <li *ngFor="let cliente of clienti">{{cliente.nome}}</li>  
</ul>
```

- Binding di proprietà

```
<img [src]="img.src" [alt]="img.description" >
```

- Binding di eventi

```
<button (click)="onSave()">Save</button>
```

- Binding bidirezionale

```
<input [(value)]="154">
```



React

- React gestisce interpolazione, proprietà e eventi, ma non ha un modo semplice per fare binding bidirezionale
- React usa il concetto di hooks (*ganci, attacchi*) per realizzare collegamenti tra parti dell'applicazione, tra cui data binding
- Sono introdotti dalla keyword `useXxx()`

- `useState()`: definisce un valore permanente modificabile con un updater:

```
const [numbers, updateNumbers] = React.useState([]);  
const onClick = () => {  
  updateNumbers([...numbers, Math.random()]);  
}
```

- `useContext()`: permette di passare valori su e giù per la catena di componenti senza passaggio esplicito negli attributi dei componenti (*props*)

```
function Button() {  
  const theme = useContext(ThemeName); //defined elsewhere  
}
```

- `useRef()`: come `useState()` definisce un valore permanente, ma che non ho bisogno di visualizzare (non viene richiesto `render()` se cambia il valore)



React - useEffect

- `useEffect()` è sostanzialmente un effetto collaterale di un'altra funzione, eseguito dopo ogni `render()`.

```
function MyButton() {  
  useEffect(() => { // Codice eseguito dopo ogni render });  
  return <button />;  
}
```

- Se non c'è un secondo parametro viene eseguito solo una volta all'inizializzazione del componente.
- Ma ***se c'è un secondo parametro***, questo indica le variabili che scatenano l'esecuzione del side effect in questione

```
function NumberList () {  
  const [numbers, updateNumbers] = React.useState([]);  
  const [counter, updateCounter] = React.useState(0);  
  React.useEffect(() => {  
    if (numbers) updateNumbers([...numbers, counter]);  
  }, [counter]);  
  const incrementCounter = () => { updateCounter(counter + 1); }  
}
```

Vue

- Vue gestisce data binding *out of the box*.
- In questo esempio, ogni volta che viene chiamato increment() cambia count **e quindi** cambia anche il contenuto dello

```
<script setup>
  const count = ref(0) ;
  function increment() { count.value++ }
</script>
<template>
  <span class="counter">{{ count }}</span>
</template>
```

- Ogni componente gestisce
 - il suo stato (valori persistenti durante la vita dell'app)
 - la sua vista (una parte del DOM in cui compare il contenuto dello stato)
 - le sue azioni (funzioni che controllano come cambia lo stato sulla base di azioni dell'utente attivate nella vista)
- Invece non è *out of the box* quando ho più viste per lo stesso stato. Usiamo la *reactivity*.



Vue – Reactivity API

- Quando lo stesso valore deve interagire con più componenti diversi lo debbo dichiarare come reattivo

```
// store.js
import { reactive } from 'vue';
export const store = reactive({
  count: 0
})
```

```
<!-- ComponentA.vue -->
<script setup>
  import { store } from './store.js'
</script>
<template>
  <span>Io sono A: {{ store.count }}</span>
</template>
```

```
<!-- ComponentB.vue -->
<script setup>
import { store } from './store.js'
</script>
<template>
  <span>Io sono B: {{ store.count }}</span>
</template>
```



Browser: Mutation observer

- Mutation observer è un oggetto del browser che può essere attivato in modo da controllare sistematicamente se qualche nodo del DOM è stato modificato (aggiunto, rimosso, cambiato di contenuto)
- Molto utile nel caso in cui io non abbia un controllo centralizzato di tutti i cambiamenti del DOM, ma voglia comunque fare qualcosa ogni volta che un componente diverso cambia qualcosa.
- Supponiamo ad esempio che io abbia svariate richieste AJAX in parallelo che a volte ma non sempre cambiano qualche parte del main del documento.

```
let observer = new MutationObserver(mutations => {  
  for(let mutation of mutations) {  
    for(let node of mutation.addedNodes) {  
      if (node.matches('.special')) {  
        node.classList.add("animate")  
      }  
    }  
  }  
});
```

```
let main = document.querySelector('main');  
observer.observe(main, {childList: true, subtree: true});
```

Signal (Angular)

- Deve l'origine ad una analogia coi sistemi di simulazione di circuiti digitali, dove l'evento scatenante è chiamate segnale.
- Basato sul pattern *observer*, un design pattern classico.
- Un segnale è un valore osservabile dotato di stato iniziale e di una funzione di aggiornamento. Quando il valore viene modificato, emette un segnale che raggiunge tutti i componenti che ne dipendono (*subscribers*) secondo un albero di dipendenze molto efficiente.
- La creazione di un segnale richiede identificare una funzione di accesso al valore (*getter*) e una funzione di modifica del valore (*setter*):

```
const val = signal(initial); val.set(newVal);
```

getter (lo invoco con `val()`)

setter (lo invoco con `setVal(nuovo valore)`
oppure con `val.set()`)

Signal (Angular)

```
import { signal } from '@angular/core';

const count = signal(0);           // getter:
count()

console.log('The count is: ' + count());

count.set(3);                      // setter:
count.set(newVal)

setInterval(() => count.update(c => c + 1), 1000);
```

- `signal(N)` crea un signal, che è sia un oggetto con metodi sia una funzione.
- `N` è il valore di inizializzazione del signal.
- Usato come funzione restituisce il valore attuale del segnale.
- I metodi `set()`, con un nuovo valore, e `update()`, con una funzione di aggiornamento, permettono di modificare il valore.



Conclusioni

- Routing e binding sono probabilmente gli aspetti più controversi e meno stabilizzati del Web odierno.
- Il routing serve per ingannare il browser rappresentando aggiornamenti di stato come navigazione. Va fatto dalla app, non dal browser.
- Next.js ha successo per via della progressiva insofferenza verso i tempi di startup troppo lunghi per l'inizializzazione ricorsiva di tutti i componenti, virtual DOM e altre strutture dati gestite internamente dalle librerie. Non è una soluzione, ma solo un cerotto temporaneo.
- Il binding serve per creare oggetti che allineano lo stato interno dell'applicazione e dei componenti con le view sul DOM.
- `useEffect()` e `signal()` sono due approcci diversi e tutt'altro che naturali per garantire questo allineamento.
- Le soluzioni divergono e non si vede né un approccio che prende il sopravvento né il WhatWG che prende posizione e ne implementa una.
- Non scommetto 10 centesimi che queste soluzioni durino nel tempo o assomiglino alle scelte che considereremo definitive.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Dipartimento di Informatica – Scienze e Ingegneria
Alma mater – Università di Bologna

Fabio.vitali@unibo.it

www.unibo.it